TP2 - Flujos de trabajo colaborativo

Materia: Metodología de Sistemas II
Unidad 4: Repositorios, colaboración y entrega continua
Alumno: Nicolás Viruel

Parte 1: Estrategias de branching

1. GitFlow vs Trunk-based

Factor — GitFlow — Trunk-based
Comunicación — Más jerárquica: roles (release manager, hotfix), ramas largas (develop, release). — Sincronizada: integración frecuente a main; acuerdos diarios.
Deuda de integración — Alta si las features viven semanas aisladas en ramas. — Baja: merges pequeños y continuos.
Conflictos — Riesgo de merge hell al acercar release o al fusionar ramas viejas. — Conflictos chicos y frecuentes, más fáciles de resolver.

2. Caso software médico (releases trimestrales, equipo grande)

Recomiendo GitFlow (o variante con ramas release/ auditables): permite congelar una release, documentar QA/regulatorio en rama estable, mantener main/develop con hotfixes controlados y trazabilidad para auditorías. Trunk-based puro encaja mejor en entrega continua diaria, no en ciclos de 3 meses con gate formal.

3. Feature flags

El código de la funcionalidad incompleta viaja en main, pero la ruta/UI permanece apagada en configuración (flag off en prod). Así se integra a diario sin exponer comportamiento inestable; en staging se prueba con flag on.

Parte 2: Integración colaborativa

4. Elementos indispensables en un PR

• Qué cambia y por qué (issue/ticket enlazado).
• Cómo probarlo (pasos, datos de prueba, capturas si aplica).
• Impacto (API, BD, permisos, performance).
• Riesgos y plan de rollback.
• Checklist DoD (tests, lint, docs).

Así el revisor entiende el contexto antes de leer el diff.

5. Code review más allá de sintaxis

Objetivos: transferencia de conocimiento, alinear criterios de diseño, detectar deuda temprana y cumplir DRY (no duplicar lógica) y SOLID (responsabilidades claras, extensiones sin romper contratos). Mejora calidad sistémica del equipo, no solo del archivo.

6. Merge commit vs Squash vs Rebase merge

Estrategia — Grafo — Cuándo
Merge commit — Historial verídico (ramas visibles). — Equipos que valoran trazabilidad de feature completa.

Squash and merge — Un commit limpio en main. — Features con muchos commits “ruidosos”.
Rebase and merge — Historial lineal sin merge commit. — Proyectos que exigen línea recta en main (cuidado con reescritura).

Parte 3: Estandarización

7. CONTRIBUTING.md

Debe incluir: requisitos de entorno (versiones), cómo clonar e instalar, convención de ramas y commits, cómo correr tests/lint, flujo de PR y revisión, contacto del maintainer, código de conducta y enlaces a DoD/estilo.

8. ADR (Architecture Decision Record)

Documenta contexto, decisión y consecuencias de una elección técnica (p. ej. “PostgreSQL vs Mongo”). Un dev que entra un año después entiende por qué no se eligió otra opción y qué trade-offs aceptó el equipo, evitando reabrir debates cerrados sin evidencia.

9. Plantillas de Issues y PRs

Homogeneizan la información mínima, reducen idas y vueltas, facilitan triage automático (labels) y métricas (tipo de trabajo, severidad).

Parte 4: Git Hooks

10. Husky y validación local

Un Git hook (p. ej. pre-commit con Husky) ejecuta lint/format/tests rápidos antes de crear el commit. Es la primera línea de defensa: el error se corrige en la máquina del autor sin contaminar el remoto ni gastar minutos de CI. Validar localmente acorta el ciclo de feedback y protege la rama compartida.